잔향

JANHYANG

제작보고서

지금 이 노래를 듣는 사람이 바로 옆에 있다는 사실

지금 듣는 곡과 음악 취향을 지도에 올리고, 같은 음악을 좋아하는 주변 사람을 발견해 대화까지 이어지는 위치 기반 음악 소셜 플랫폼

과목
포스트캡스톤

제출일
2026. 11. 6.

서비스 주소
soundmap.duckdns.org

저장소
github.com/HaeseokPark/Soundmap

구분
반응형 웹 (PWA · 설치형)

제출자
박해석

51	19	48	11,642
REST 엔드포인트	도메인 엔티티	자동화 테스트	소스 라인

Java 21 · Spring Boot 4 · React 19 · MySQL 8 · AWS

01 문제 정의

PROBLEM

음악 취향은 사람에 대해 가장 많은 것을 말해 주는 신호다. 그런데 지금의 서비스 지형에서는 **그 신호가 바깥으로 전혀 새어 나가지 않는다.** 스트리밍 앱은 혼자 듣기에 최적화되어 있고, SNS는 이미 아는 사람과만 연결한다. 결과적으로 같은 캠퍼스, 같은 동네에서 같은 노래를 듣는 사람들이 서로의 존재를 끝까지 모른 채 지나간다.

세 가지 공백

GAP 01

취향은 있는데 접점이 없다

스트리밍 서비스는 "이 곡을 좋아한 사람이 들은 곡"은 추천하지만 **그 사람 자체**는 끝내 보여주지 않는다. 취향 데이터가 추천 알고리즘 안에서만 소비되고 관계로 이어지지 않는다.

GAP 02

위치 기반 서비스가 무섭다

주변 사람을 찾아 주는 서비스는 많지만, 대부분 **정확한 좌표**를 올린다. 내가 지금 어디 있는지가 그대로 노출된다는 불안이 진입 자체를 막는다.

GAP 03

낯선 사람과 말 트기가 어렵다

연결은 쉬운데 **첫 문장**이 어렵다. 공통점이 없는 상태에서 열린 메시지 창은 금세 일방적 접근과 괴롭힘의 통로가 된다.

그래서 잔향이 세운 설계 원칙

① 대화의 출발점을 사람이 아니라 곡으로 둔다

"안녕하세요"로 시작하지 않는다. 지도 위 마커는 사람 얼굴이 아니라 **그 사람이 지금 듣고 있는 앨범 아트**다. 프로필을 열면 먼저 보이는 것은 나이도 사진도 아니라 "두 분 모두 폴킴을 좋아합니다"라는 한 줄이다. 할 말을 서비스가 미리 만들어 준다.

② 위치는 흐리게 쓴다 — 원본 GPS는 저장하지 않는다

좌표를 받으면 서버는 그 즉시 약 100m 격자로 스냅하고, **스냅된 값만 DB에 넣는다.** 원본은 메모리에서 사라진다. 같은 자리에서 몇 번을 갱신해도 항상 같은 격자점이 나오기 때문에, 여러 번 받아 평균을 내는 방식으로 실제 위치를 역산하는 것도 막힌다.

③ 대화는 양쪽이 동의해야 열린다

수락 전에는 메시지를 한 글자도 보낼 수 없다. 거절당한 뒤 다시 요청하는 것도 막는다. 차단은 한쪽만 눌러도 양방향으로 적용되며, 차단당한 쪽에게는 그 사실이 드러나지 않는다.

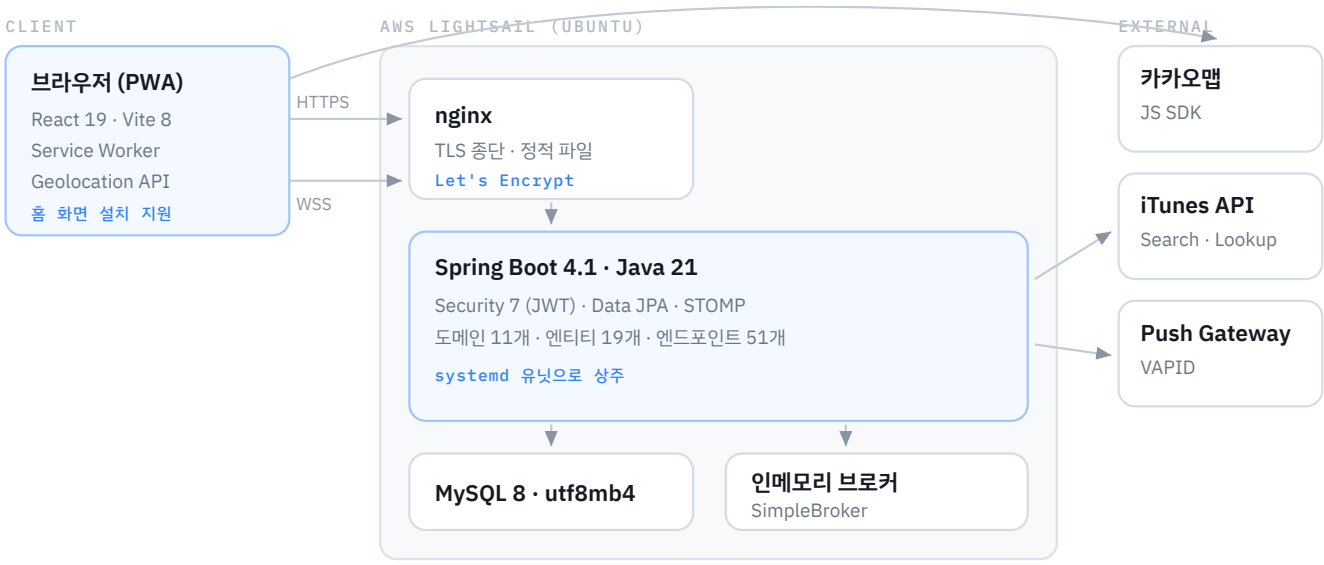
목표 사용자

대상	상황	잔향이 주는 것
대학 캠퍼스 학생	같은 건물에 매일 수백 명이 있지만 접점이 수업뿐이다	취향이 겹치는 사람을 거리순·일치도순으로 발견
음악 취향이 뚜렷한 사람	주변에 같은 음악을 아는 사람이 없다고 느낀다	반경을 전국까지 넓혀 취향으로만 탐색
새 음악을 찾는 사람	알고리즘 추천이 비슷한 곡만 돌려준다	실제 사람이 오늘 공유한 곡 — 동네 차트

02 시스템 구조와 기술 스택

ARCHITECTURE

프런트엔드는 정적 번들로 빌드해 nginx 가 서비스하고, 같은 도메인의 `/api` 와 `/ws` 를 Spring Boot 로 넘긴다. 같은 출처를 쓰기 때문에 CORS 와 혼합 콘텐츠 문제가 없고, WebSocket 도 현재 주소에서 그대로 유도된다.

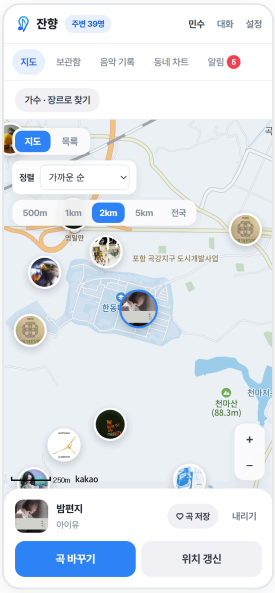


지도 SDK 만 브라우저가 직접 호출한다. iTunes 와 푸시 게이트웨이는 서버만 접근하므로 키가 노출되지 않는다.

기술 스택과 선택 이유

영역	선택	왜 이것인가
백엔드	**Java 21 · Spring Boot 4.1.1**	도메인이 11개로 갈라지는 구조라 계층과 트랜잭션 경계가 분명한 쪽이 유리했다. 레코드·패턴 매칭으로 DTO 가 가벼워진다.
인증	**JWT (HS256, JJWT 0.13)**	세션 저장소 없이 수평 확장이 가능하다. WebSocket 핸드셰이크에 헤더를 붙일 수 없어 STOMP CONNECT 프레임으로 토큰을 넘긴다.
DB	**MySQL 8 (utf8mb4)**	한국어·이모지를 온전히 담아야 한다. PostGIS 가 없는 제약은 바운딩박스 + haversine 2단계로 풀었다(8장).
실시간	**STOMP over WebSocket**	1:1 채팅 규모에서 외부 브로커는 과하다. Spring 내장 SimpleBroker 로 운영 요소를 하나 줄였다.
지도	**카카오맵 JS SDK**	국내 건물·도로 데이터가 압도적이다. 기본 마커로는 앨범 아트를 원형으로 자를 수 없어 CustomOverlay 에 DOM 을 직접 넣었다.
음악 메타	**iTunes Search API**	인증 없이 쓸 수 있고 앨범 아트와 30초 미리듣기를 함께 준다. 한국 스토어 제약은 US 검색 + KR lookup 으로 우회했다(8장).
알림	**Web Push (VAPID)**	앱 설치 없이 브라우저만으로 푸시를 받는다. 서버 키는 환경변수로만 주입하고 저장소에 두지 않는다.
프런트	**React 19 · Vite 8**	지도·시트·목록이 같은 상태를 공유해 선언형이 맞는다. Vite 의 HMR 로 지도 작업의 반복 주기가 짧다.
배포	**Lightsail · nginx · systemd**	고정 비용으로 상시 가동한다. Geolocation API 가 보안 컨텍스트를 요구해 HTTPS 가 선택이 아닌 필수였다.

모든 화면은 실제 동작하는 빌드를 390×844 (iPhone 기준)로 촬영한 것이다. 지도 위 인물은 한동대학교 반경 1.5km 안에 배치된 개발용 더미 사용자 40명이다.



FEATURE 01

앨범 아트가 곧 마커인 지도

지도 위의 동그라미는 사람 사진이 아니라 **그 사람이 지금 듣고 있는 앨범 커버**다. 화면을 훑는 것만으로 "이 동네가 지금 무슨 음악을 듣고 있는지"가 읽힌다. 내 마커만 강조색 테두리에 조금 더 크게 그려 한눈에 찾을 수 있다.

반경은 500m부터 전국까지 다섯 단계다. 전국은 상대가 수백 km 밖이라 지도에서 화면을 벗어나므로 자동으로 목록 보기로 넘긴다.

구현 카카오맵 CustomOverlay 에 직접 만든 DOM 삽입 · 30초 주기 자동 갱신(숨은 탭에서는 건너뜀) · ResizeObserver 로 컨테이너 크기 변화 시 `relayout()`

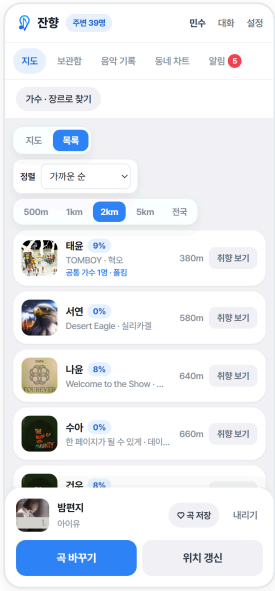
FEATURE 02

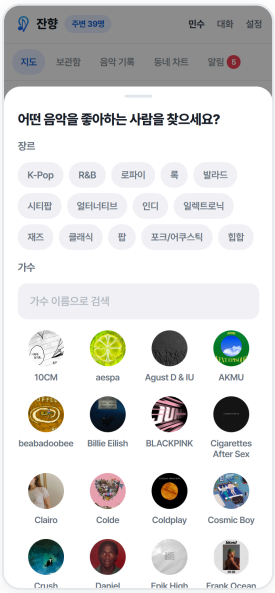
목록 보기와 취향 일치도

지도와 목록은 **같은 한 번의 응답**을 쓴다. 화면에서만 갈라 쓰기 때문에 두 보기가 어긋나지 않는다. 각 줄에는 거리, 듣고 있는 곡, 그리고 **취향 일치도(%)**가 붙는다.

정렬은 **가까운 순**과 **취향이 비슷한 순** 둘이다. 일치도는 자카드 유사도로 계산해 취향의 폭이 점수를 부풀리지 못하게 했다(8장). 공통 가수가 있으면 "공통 가수 1명 · 폴킴"처럼 이름까지 보여 준다 — 그게 곧 첫 문장이 된다.

구현 `GET /api/map/pins` 하나로 지도·목록 공용 · 가수/장르 필터는 애플리케이션 단에서 집합 연산 · 마커가 4개든 42개든 **쿼리 수 9개 고정**





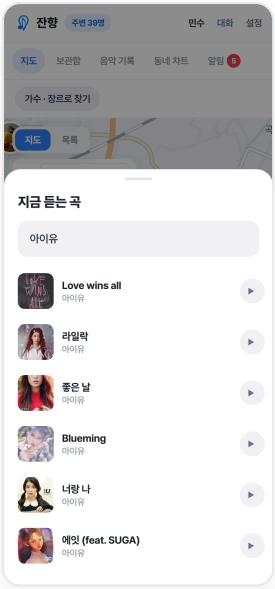
FEATURE 03

가수 · 장르로 좁혀 찾기

"어떤 음악을 좋아하는 사람을 찾으세요?" 한 줄로 시작한다. 장르 14개 중 하나를 누르거나 가수 58명 중에서 고르면, 지도와 목록이 **그 취향을 가진 사람만** 남긴다.

가수는 이름으로 검색할 수 있고, 대표곡 아트워크를 얼굴로 쓴다. 이미지는 시딩할 때 한 번 받아 저장소에 굳혀 두어 매번 외부 API 를 부르지 않는다.

구현 즐겨찾기 테이블은 `(user_id, artist_id)` 복합 PK — 중복 저장을 구조로 차단하고 PK 인덱스를 조회에 그대로 재사용 · 역방향 조회를 위해 `artist_id` 별도 인덱스



FEATURE 04

지금 듣는 곡 올리기

제목이나 가수 이름을 넣으면 iTunes 에서 곡을 찾아 준다. 고르는 순간 내 마커의 앨범 아트가 바뀌고, 같은 반경 안의 다른 사람 화면에도 반영된다. 옆의 재생 버튼으로 **30초 미리듣기**를 들어 볼 수 있다.

화면에서 보이듯 **한국어로 검색하면 한국어 제목이 나온다.** 간단해 보이지만 iTunes 한국 스토어는 곡 검색 결과를 아예 주지 않는다. 미국 스토어로 찾은 뒤 받은 ID 를 한국 스토어에 다시 조회해 제목만 바꿔 끼우는 방식으로 풀었다(8장).

구현 검색 결과 30분 · 최대 300건 캐싱(분당 20회 쿼터 대응) · 빈 결과는 캐시하지 않음 — 레이트 리밋으로 비어 온 검색어가 30분간 막히는 것을 막기 위해

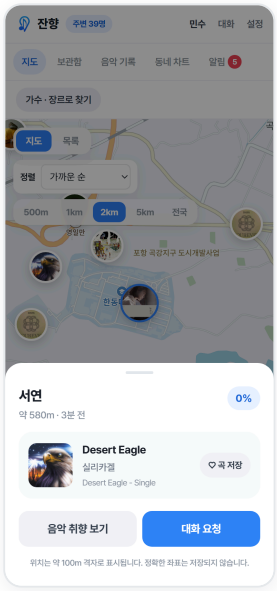
FEATURE 05

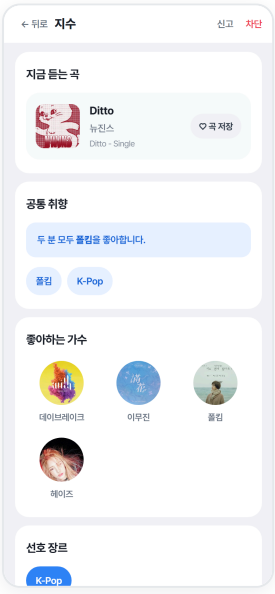
핀을 열면 보이는 것

마커를 누르면 닉네임, **대략적인 거리**, 마지막 갱신 시각, 듣고 있는 곡, 그리고 취향 일치도가 한 장에 담긴다. 여기서 바로 곡을 보관함에 저장하거나 **대화 요청**을 보낼 수 있다.

맨 아래 한 줄을 눈여겨볼 만하다 — **"위치는 약 100m 격자로 표시됩니다. 정확한 좌표는 저장되지 않습니다."** 프라이버시 정책을 약관에 묻어 두지 않고, 사용자가 불안을 느끼는 바로 그 화면에서 말한다.

구현 거리는 서버에서 haversine 으로 계산해 내려줌 · 차단한 상대는 SQL 단계에서 제외되어 응답 자체에 존재하지 않음 (개발자 도구로도 보이지 않는다)





FEATURE 06

공통 취향이 먼저 보이는 프로필

프로필을 열었을 때 가장 먼저 눈에 들어오는 것은 나이도 사진도 아니라 **"두 분 모두 폴킴을 좋아합니다"**라는 한 문장이다. 그 아래로 공통 가수와 장르를 칩으로 늘어놓고, 그다음에야 상대의 전체 취향이 나온다.

낯선 사람에게 말 거는 일의 어려움은 대부분 **첫 문장**에 있다. 그 문장을 서비스가 대신 만들어 주는 것이 이 화면의 목적이다.

구현 조사 처리기(`objectParticle`)로 받침에 따라 을/를 을 고른다 — 그냥 붙이면 "폴킴를 좋아합니다"가 된다 · 공개 범위가 `CHAT_ONLY` 면 수락된 상대에게만 취향을 보여준다



FEATURE 07

수락해야 열리는 대화

대화는 **요청 → 수락**을 거쳐야 시작된다. 수락 전에는 메시지를 한 글자도 보낼 수 없다. 받은 요청에는 상대의 취향 일치도가 함께 표시되어, 수락 여부를 판단할 재료가 된다.

요청자가 자기 요청을 스스로 수락하는 것도, 거절당한 뒤 다시 요청하는 것도 막았다. 낯선 사람이 일방적으로 말을 걸 수 있으면 위치 기반 서비스에서 그건 곧 괴롭힘 통로가 된다.

구현 방은 두 사용자 id 를 작은 쪽/큰 쪽으로 정렬해 저장 — A→B 와 B→A 가 같은 행으로 모여 **유니크 제약 하나로 중복 방을 차단**

FEATURE 08

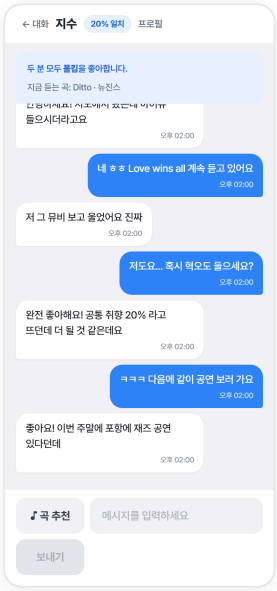
곡을 주고받는 1:1 채팅

WebSocket(STOMP) 기반 실시간 채팅이다. 상단에는 공통 취향 배너와 상대가 지금 듣는 곡이 고정되어, 대화 중에도 "무슨 이야기를 할지"가 계속 눈앞에 있다.

입력창 옆 **곡 추천** 버튼으로 곡을 카드 형태로 보낼 수 있다. 받은 카드는 바로 미리듣기와 보관함 저장으로 이어진다 — 음악으로 만난 사이의 대화가 다시 음악으로 돌아온다.

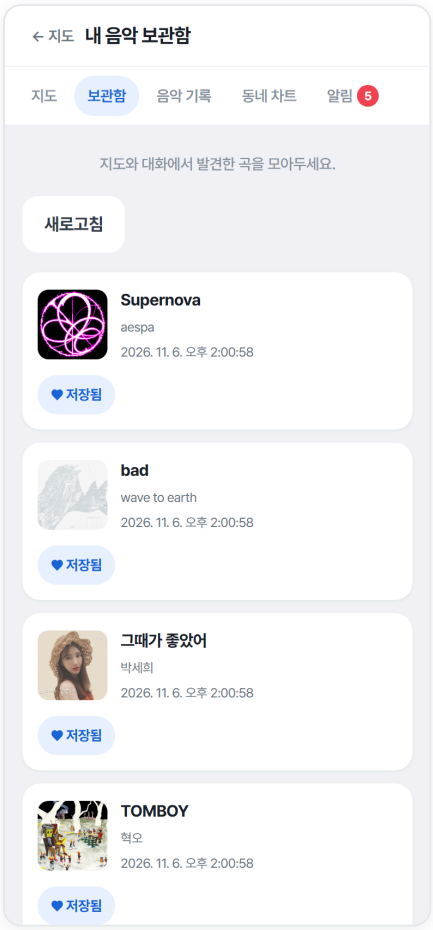
위로 당기면 이전 메시지를 50건씩 더 불러오며, 이때 읽던 위치를 유지하도록 스크롤 높이 차이만큼 되돌려 놓는다.

구현 전송 시 방 행에 **쓰기 잠금**(`findByIdForUpdate`) — 없으면 동시 전송에서 안 읽은 수가 사라진다(8장) · WebSocket 이 끊겨도 REST 로 보낼 수 있게 두 경로를 모두 열되 검증과 저장은 같은 서비스 메서드를 거친다

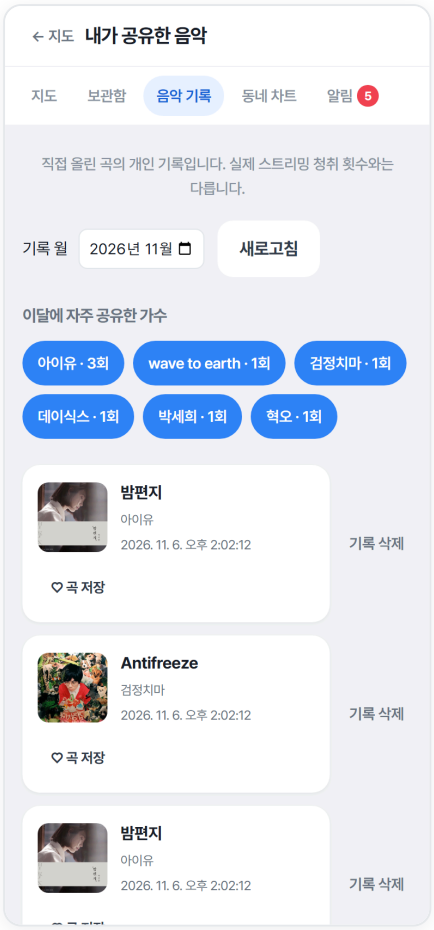


06 주요 기능 — 음악 발견

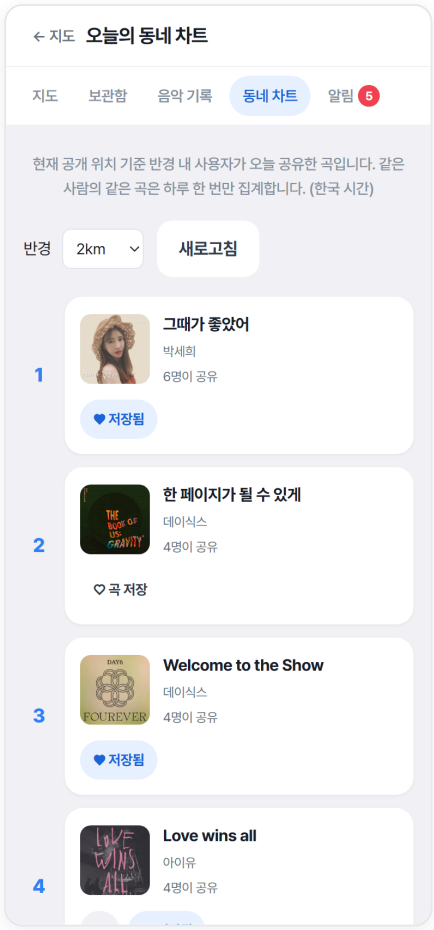
지도에서 만난 음악이 흘러가 버리지 않도록, 모으고 돌아보고 받아 보는 세 가지 화면을 더했다.



보관함
지도·채팅에서 발견한 곡을 모은다



음악 기록
월별로 돌아보는 내 공유 이력



동네 차트
오늘 이 근처에서 많이 공유된 곡

FEATURE 10

내 음악 보관함

지도에서 스쳐 본 곡, 대화에서 받은 곡 카드를 하트 하나로 모은다. 저장 버튼은 보관함 상태를 전역으로 들고 있어서, 어느 화면에 있든 **저장됨** 상태가 즉시 반영된다.

FEATURE 11

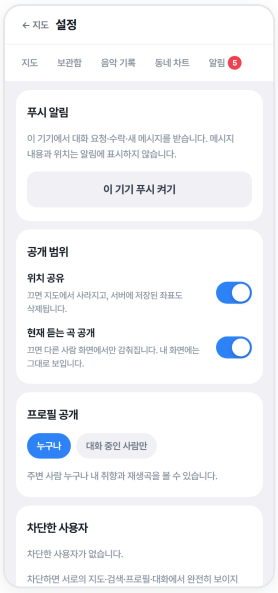
내가 공유한 음악

월을 고르면 그달에 올린 곡과 **"이달에 자주 공유한 가수"** 요약이 나온다. 스트리밍 청취 횟수가 아니라 **내가 남에게 보여준 음악**의 기록이라는 점을 화면에서 분명히 밝힌다.

FEATURE 12

오늘의 동네 차트

반경 안의 사람들이 **오늘** 공유한 곡을 집계한다. 같은 사람의 같은 곡은 하루 한 번만 세어 한 명이 순위를 밀어 올리지 못하게 했다. 알고리즘이 아니라 **실제 옆 사람**이 만든 차트다.



FEATURE 09

끌 수 있어야 켤 수 있다 — 공개 범위

위치 공유, 현재 듣는 곡 공개, 프로필 공개를 각각 끌 수 있다. 중요한 것은 **끄는 방식이 항목마다 다르다**는 점이다.

- **위치 공유**를 끄면 저장된 좌표를 **삭제**하고 새 좌표도 받지 않는다. "어디 있는지"는 서버가 아예 보관하지 않는 편이 안전하다.
- **현재 듣는 곡**은 남의 화면에서만 감춘다. 내 화면에는 그대로 보인다. 이건 "무엇을 보여줄지"의 문제라 내 쪽에는 계속 필요하다.
- **프로필 공개**를 `대화 중인 사람만`으로 두면 수락된 상대에게만 취향이 보인다.

구현 설정 행은 한 번이라도 건드린 사용자에게만 생기고, 없으면 기본값(전부 공개)으로 해석 · 차단 목록도 이 화면에서 해제할 수 있다

차단은 한쪽만 눌러도 양방향으로 동작한다

A가 B를 차단하면 **B의 지도에서도 A가 사라진다.** 차단당한 쪽에게 그 사실이 드러나지 않아야 하고, 계정을 새로 만들지 않는 한 다시 접근할 수 없어야 하기 때문이다. 차단은 **지도 · 검색 · 프로필 · 채팅 목록 · 메시지 전송 · 대화 요청** 여섯 군데 모두에 걸리는데, 걸러내는 곳이 흩어지면 한 곳을 빠뜨렸을 때 그게 곧 구멍이 된다. 그래서 `SafetyService.hiddenUserIdsFor` 한 곳에서 집합을 만들어 각 화면이 가져다 쓴다. 차단된 상대의 프로필은 403 이 아니라 **404** 를 준다 — 403 은 "차단당했구나"를 알려주는 셈이다.



FEATURE 13

알림함과 선택형 웹 푸시

대화 요청, 수락, 새 메시지가 알림함에 쌓인다. 안 읽은 알림은 배경을 띄워 구분하고 상단 메뉴의 배지로 개수를 보여 준다.

푸시는 기기마다 따로 켠다. 앱 설치 없이 브라우저만으로 받을 수 있고, 알림에는 **메시지 내용과 위치를 담지 않는다** — 잠금 화면은 남이 볼 수 있는 공간이기 때문이다. 서버에 푸시 키가 없어도 알림함 자체는 정상 동작하도록 두 기능을 분리했다.

구현 VAPID 키쌍으로 서명(nl.martijndwars:web-push + BouncyCastle) · 구독 정보는 기기 단위 저장 · 개인키는 환경변수로만 주입하고 저장소에 올리지 않는다

08 기술적 특장점

ENGINEERING

기능 목록만으로는 드러나지 않는, **이 서비스를 실제로 돌아가게 만든 판단**들을 모았다. 여섯 가지 모두 "그냥 되는" 문제가 아니라 한 번씩 막혔다가 풀어낸 것이다.

① 위치 프라이버시 — 평균을 내도 역산되지 않는 격자

좌표를 받으면 서버는 **저장 전에** 약 100m 격자로 스냅한다. 원본 GPS 값은 DB 에 들어가지 않는다. 중요한 것은 **결정론적**으로 스냅한다는 점이다 — 같은 자리에서 백 번을 갱신해도 언제나 같은 격자점이 나온다. 매번 무작위 오차를 더하는 방식이었다면 여러 번 모아 평균을 내는 것만으로 실제 위치가 드러났을 것이다.

> `GeoGrid.snap(lat, 0.001)` — 소수점 셋째 자리로 HALF_UP 반올림. 위도 0.001도 ≈ 111m. 격자 크기는 `LOCATION_GRID_SIZE` 환경변수로 조절하며, 키울수록 익명성이 높아지고 정밀도가 낮아진다.

② MySQL 에서 반경 검색을 인덱스 위로 올리기

MySQL 에는 PostGIS 의 `ST_DWithin` 같은, 반경 검색이 인덱스를 타는 함수가 없다. `ST_Distance_Sphere(...) < 2000` 으로 쓰면 전체 스캔이 된다. 그래서 두 단계로 나눴다.

단계	무엇을	어떻게
1	사각형으로 후보 압축	`grid_lat` , `grid_lon` 에 `BETWEEN` — 복합 인덱스를 탄다
2	원형 거리로 재검증	애플리케이션에서 haversine 계산 후 가까운 순 정렬

사각형은 원보다 넓으므로 2단계가 반드시 필요하다. 이 성질을 `GeoDistanceTest.boundingBoxIsWiderThanCircle` 로 고정해 두었다.

③ 취향의 폭이 점수를 부풀리지 못하게 — 자카드 유사도

교집합 크기만 세면 가수를 30명 고른 사람이 무조건 유리해진다. 합집합으로 나누면 **"고른 것 대비 얼마나 겹치는가"**가 되어 취향의 폭과 무관해진다. 최종 점수는 **가수 70% + 장르 30%** — 장르는 14개뿐이라 우연히 겹치기 쉬운 반면, 같은 가수를 좋아한다는 건 훨씬 구체적인 신호이기 때문이다.

이 성질도 테스트로 박아 두었다 — `TasteMatchTest.breadthDoesNotInflateScore` .

④ 마커가 몇 개든 쿼리는 고정 — N+1 차단

재생곡을 사용자 수만큼 조회하지 않고 `IN` 절로 한 번에 가져온다. **마커가 4개든 42개든 지도 한 번 그리는 데 쿼리 9개로 고정**이다. 가수/장르 필터를 SQL 이 아니라 애플리케이션에서 거는 것도 같은 이유다 — 일치도 계산을 위해 후보들의 취향을 어차피 한 번에 불러오므로, 필터링이 집합 조회로 끝나 추가 쿼리가 들지 않는다.

⑤ 동시 전송에서 사라지는 카운터 — 잠금으로 막은 lost update

안 읽은 개수와 마지막 메시지는 방 목록을 그릴 때마다 메시지 테이블을 뒤지지 않도록 `chat_rooms` 행에 함께 둔다. 그런데 메시지가 거의 동시에 들어오면 두 트랜잭션이 같은 값을 읽고 같은 값을 써서 증가가 사라진다.

> **실제로 확인한 것** — 3건을 연달아 보냈더니 카운터가 **1**로 남았다. 전송 시 방 행에 쓰기 잠금(`findByIdForUpdate`)을 건 뒤 **20건 동시 전송에서 정확히 20**이 나오는 것을 확인했다.

⑥ 한국 스토어가 곡을 주지 않는 문제 — 2단 조회

처음에는 곡 검색 결과가 늘 비어 있었다. 원인은 코드가 아니라 **iTunes 한국 스토어프론트가 `entity=song` 검색에 항상 0건을 돌려준다**는 사실이었다 — 한국은 iTunes 음악 판매 지역이 아니다. 그렇다고 미국 스토어만 쓰면 제목이 전부 로마자로 나온다("Bam Pyeon Ji").

> **해법** 곡은 **US** 스토어로 찾고, 받은 `trackId` 묶음을 **KR** `/lookup` 으로 한 번 더 조회해 **제목과 가수명만 한국어로 바꿔 끼운다.** 앨범 아트와 미리듣기 주소는 US 응답을 그대로 쓴다(KR 에는 미리듣기가 없다). 조회가 한 번 더 늘지만 ID 묶음 하나로 끝나므로 곡 수에 비례하지 않는다. `ITUNES_LOCALIZE_COUNTRY` 를 비우면 이 단계는 꺼진다.

그 밖에

- **한국어 조판** — `word-break: keep-all` 이 없으면 "다릅니다"가 "다릅 / 니다"로 갈라진다. 다만 그것만 두면 긴 주소가 화면 밖으로 나가므로 `overflow-wrap: break-word` 를 함께 건다.
- **위치 정확도 샘플링** — `getCurrentPosition` 은 가장 먼저 도착한 거친 값을 준다. `watchPosition` 으로 최대 5초간 지켜보며 가장 정확한 값을 고르고, 오차 20m 이하면 즉시 확정, 1km 초과면 저장을 거부한다.
- **테스트로 성질 고정** — 48개 테스트 중 상당수가 "버그가 없다"가 아니라 **"이 성질이 유지된다"**를 검증한다(바운딩박스 ⊃ 원, 취향 폭 비의존성, 동시성).
- **PWA** — 매니페스트·서비스워커·마스커블 아이콘을 갖춰 홈 화면에 설치하면 전체화면 앱처럼 실행된다. iOS 웹 푸시도 설치 상태에서만 동작한다.

사용자에게

EFFECT 01

말 걸기의 비용을 낮춘다

"안녕하세요" 대신 **"두 분 모두 폴킴을 좋아합니다"**에서 출발한다. 공통점을 서비스가 먼저 찾아 주기 때문에, 낯선 사람과의 첫 대화에 드는 심리적 비용이 눈에 띄게 줄어든다.

EFFECT 02

불안 없이 위치를 쓴다

100m 격자, 원본 미저장, 언제든 끄면 즉시 삭제. **"위치를 쓰는 서비스는 무섭다"**는 기본 전제를 설계로 뒤집는다. 위치 기반 서비스의 가장 큰 진입 장벽을 낮추는 접근이다.

EFFECT 03

알고리즘 바깥의 음악

추천 엔진은 내가 이미 좋아한 것과 비슷한 곡을 준다. 동네 차트는 **실제 옆 사람이 오늘 들은 곡**이라, 내 취향 바깥이지만 같은 공간을 쓰는 사람들의 음악이 들어온다.

캠퍼스 · 지역 단위로

반경 500m~2km 라는 기본 설정은 **캠퍼스 하나, 동네 하나**의 크기다. 같은 건물에서 매일 마주치지만 수업 말고는 접점이 없던 사람들 사이에 "음악"이라는 공통 화제를 하나 놓는다. 사용자가 일정 밀도 이상 모이면 동네 차트는 **그 공간의 정서를 보여주는 지표**가 된다 — 시험 기간의 플레이리스트와 축제 주간의 플레이리스트는 분명히 다를 것이다.

기술적으로

확보한 것	재사용 가능성
위치 익명화 패턴	결정론적 격자 스냅은 배달·중고거래·모임 등 "대략 어디" 수준이면 충분한 모든 위치 서비스에 그대로 쓸 수 있다.
인덱스 친화적 반경 검색	PostGIS 없이 MySQL 만으로 주변 검색을 구현해야 하는 상황에서 바로 적용 가능한 2단계 구조.
취향 일치도 모델	자카드 유사도 + 가중 합성은 음악 외에 영화·도서·관심사 매칭에도 같은 형태로 옮겨진다.
안전 장치의 단일 소스	차단 집합을 한 곳에서 만들어 전 화면이 공유하는 구조는, 조회 지점이 늘어나는 모든 소셜 서비스의 기본기다.

현재 상태

> **실제로 접속해 쓸 수 있는 상태다.** AWS Lightsail 에 상시 배포되어 있고 (`soundmap.duckdns.org`), Let's Encrypt 인증서로 HTTPS 가 걸려 있으며, systemd 로 상주한다. 휴대폰 브라우저로 접속해 홈 화면에 추가하면 앱처럼 전체화면으로 실행된다. **데모 환경이 아니라 운영 환경**이라는 점이 이 프로젝트의 완성도를 보여주는 가장 단순한 증거다.

10 추가 개발 계획

ROADMAP

1단계 — 네이티브 앱 전환 (최우선)

현재 가장 큰 구조적 한계는 **웹이라서 못 하는 것**들이다. 이 서비스의 성격상 둘 다 치명적이다.

한계	지금 (웹 PWA)	네이티브 전환 후
백그라운드 위치	불가능. 앱을 열고 있을 때만 위치를 안다	지나가다 취향이 겹치는 사람 근처에 왔을 때 알림 **— 이 서비스의 본래 가치**
푸시 신뢰성	iOS 는 홈 화면에 추가해야만 동작하고, 구독이 조용히 끊기는 사례가 있다	APNs 직접 사용. 설치하면 그냥 된다
유통	링크 공유뿐	앱스토어 검색·설치

전환 경로는 두 가지를 검토했다. **Flutter 재작성**은 네이티브 성능과 안드로이드 동시 지원을 얻지만 프런트 6,054라인을 전부 버려야 한다 (카카오맵은 `kakao_map_sdk` 로 네이티브 렌더링이 가능하다). **Capacitor 래핑**은 현재 React 코드를 거의 그대로 둔 채 APNs 와 백그라운드 위치만 플러그인으로 얻는다. 두 경우 모두 백엔드는 REST + STOMP 그대로라 **수정이 필요 없다.**

2단계 — 음악 연동 자동화

지금은 듣는 곡을 **직접 검색해서 올린다.** 한 번의 수고가 들어가는 만큼, "진짜 지금 듣는 곡"이 아닐 수 있다. Spotify Web API 의 currently-playing, Apple Music MusicKit 연동으로 **실제 재생 중인 곡을 자동 반영**하면 지도의 신선도가 근본적으로 달라진다.

3단계 — 밀도가 올라갈 때 필요한 것

- **마커 클러스터링** — 지금은 더미 40명 기준으로 견딜 만하지만, 실사용자가 늘면 마커가 겹친다. 카카오맵 `MarkerClusterer` 를 붙일 자리를 비워 두었다.
- **장소 기반 확장** — 카페·공연장·학과 건물 단위의 "이 장소의 지금 플레이리스트". 사람 대 사람에서 사람 대 공간으로 축을 하나 늘린다.
- **취향 변화 추적** — 음악 기록이 쌓이면 "올해 내 음악 취향이 어떻게 변했는지"를 연말 결산 형태로 보여줄 수 있다.

4단계 — 운영 성숙도

항목	현재	계획
토큰 보관	`localStorage` (XSS 노출 가능)	refresh 토큰 + HttpOnly 쿠키
가수 표기	온보딩 목록이 영문 표기 (IU, HYUKOH)	한국어 표기 컬럼 추가 — 기존 즐겨찾기를 깨지 않도록 이름이 아닌 별도 필드로
신고 처리	접수까지만 (운영자 화면 없음)	신고 누적 기준 자동 제한 + 검토 화면
관측성	systemd 저널 로그	구조화 로그 + 에러 추적 + 핵심 지표 대시보드
배포	수동 빌드·전송	GitHub Actions 로 빌드·테스트·배포 자동화

11 개발 과정에서 막혔던 것들

TROUBLESHOOTING

완성된 화면만으로는 보이지 않는 부분이다. 여기 적힌 다섯 가지는 모두 **원인이 코드가 아닌 곳에 있어서** 오래 걸렸던 문제다.

① 검색 결과가 항상 0건 — 코드가 아니라 스토어프론트

곡 검색이 늘 비어 있었다. 요청 URL, 파라미터, 파싱을 몇 번이나 들여다봤지만 전부 정상이었다. `curl` 로 직접 불러 보고서야 알았다 — **iTunes 한국 스토어프론트는 `entity=song` 검색에 항상 0건을 준다.** 한국이 iTunes 음악 판매 지역이 아니기 때문이다. 코드 어디에도 버그가 없었다.

배운 것 외부 API 가 비어 있을 때는 내 코드를 의심하기 전에 **그 API 를 직접 호출해** 응답을 눈으로 확인한다.

② 카카오맵이 안 뜨는데 콘솔에 아무 말이 없다

지도가 빈 상자로 남았다. 브라우저 콘솔에는 단서가 없었다. 서버에서 `curl` 로 SDK 를 받아 보니 응답 본문에 이유가 담겨 있었다 — `App(사운드맵) disabled OPEN_MAP_AND_LOCAL service.`

> **카카오맵 키는 세 단계를 다 통과해야 뜬다.** ① 유효한 JavaScript 키 ② 개발자 콘솔에 **사이트 도메인 등록** ③ 제품 설정에서 **카카오맵 활성화**. 하나라도 빠지면 스크립트가 조용히 실패하고, 브라우저 콘솔만 봐서는 어느 단계인지 알 수 없다.

③ 확대/축소가 한 번만 되고 멈춘다

줌 버튼이 첫 클릭만 먹고 그 뒤로 축척이 바뀌지 않았다. 격리 테스트로 두 호출을 비교해 원인을 좁혔다.

호출	네 번 눌렀을 때 레벨 변화
`setLevel(n, { animate: true })`	7 → 6 → 6 → 6 **(멈춤)**
`setLevel(n)`	7 → 6 → 5 → 4 **(정상)**

문서에 적혀 있지 않은 동작이라 추측으로는 찾을 수 없었다. 옵션을 빼는 것으로 해결.

④ 편집이 조용히 실패한다 — 섞여 있던 줄바꿈

파일을 고치는 스크립트가 찾는 문자열을 못 찾고 아무 일 없이 끝나는 일이 반복됐다. 바이트 단위로 들여다보니 한 파일 안에 `CRLF` 와 `LF` 가 섞여 있었다. Windows 에서 개발하고 리눅스에 배포하는 구조에서 생긴 문제다. 셸 스크립트가 CRLF 로 체크아웃되면 리눅스에서 **실행 자체가 되지 않는다.** `.gitattributes` 와 `.editorconfig` 로 저장소 전체를 LF 로 고정했다.

⑤ nginx 설정을 덮어썼더니 HTTPS 가 내려갔다

저장소의 nginx 설정 파일을 서버에 그대로 복사했더니 사이트가 HTTPS 로 접속되지 않았다. `nginx -t` 는 통과했기 때문에 바로 알아채지 못했다. 원인은 **certbot 이 443 리스너와 인증서 경로를 서버 파일에만 추가해 두었다**는 것 — 저장소 파일에는 그 블록이 없으니 덮어쓰는 순간 사라진다. 백업에서 복구한 뒤, 서버 파일을 직접 수정하고 `ss -lntp | grep :443` 으로 확인하는 절차로 바꿨다. 저장소 파일 상단에는 경고 주석을 달아 두었다.

> **공통점** 다섯 건 모두 **내가 쓴 코드 바깥**에 원인이 있었다. 외부 API 의 지역 정책, 콘솔의 제품 설정, 문서에 없는 SDK 동작, 운영체제 간 줄바꿈 규약, 배포 도구가 몰래 추가한 설정. 이런 문제는 코드를 다시 읽어서는 찾을 수 없고, **경계 바깥에서 직접 눈으로 확인**해야만 보인다는 것을 배웠다. 그래서 이 내용들을 전부 저장소 README 의 "알아둘 것"에 남겨 두었다.

12 부록

APPENDIX

API 구성 (전체 51개)

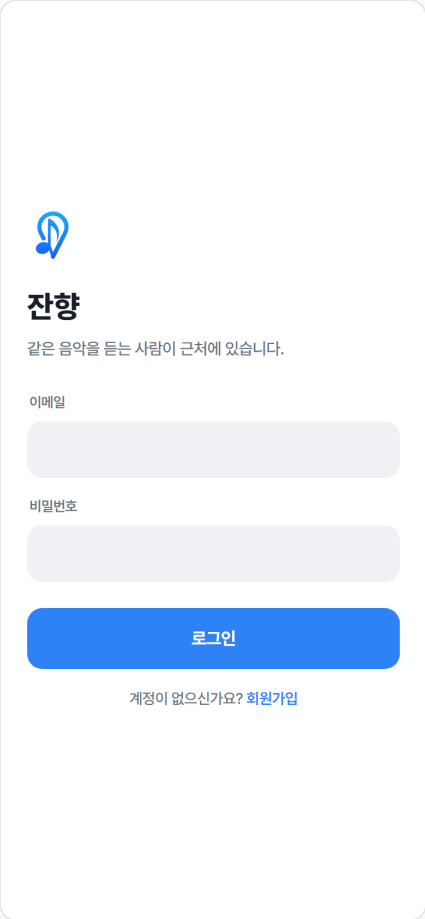
영역	개수	대표 엔드포인트
인증	5	`POST /api/auth/signup` · `/login` · `GET /api/auth/me`
음악	4	`GET /api/tracks/search` · `PUT|GET|DELETE /api/me/now-playing`
위치 · 지도	4	`PUT /api/me/location` · `GET /api/map/pins`
취향 · 프로필	5	`GET /api/artists` · `PUT /api/me/taste` · `GET /api/users/{id}/profile`
채팅	9	`POST /api/chats/requests` · `/{id}/accept` · `/{id}/messages`
안전 · 설정	7	`POST /api/blocks` · `POST /api/reports` · `PUT /api/me/settings`
음악 발견	8	`GET /api/me/saved-tracks` · `/music-history` · `/api/music/local-chart`
알림	6	`GET /api/me/notifications` · `PUT /api/me/notifications/push`

실시간은 STOMP — 발행 `/app/chats/{id}/send`, 구독 `/topic/chats/{id}` · `/user/queue/chats` · `/user/queue/errors`. 토큰은 CONNECT 프레임의 `Authorization` 헤더로 넘긴다(브라우저 WebSocket 은 핸드셰이크에 임의 헤더를 붙일 수 없기 때문).

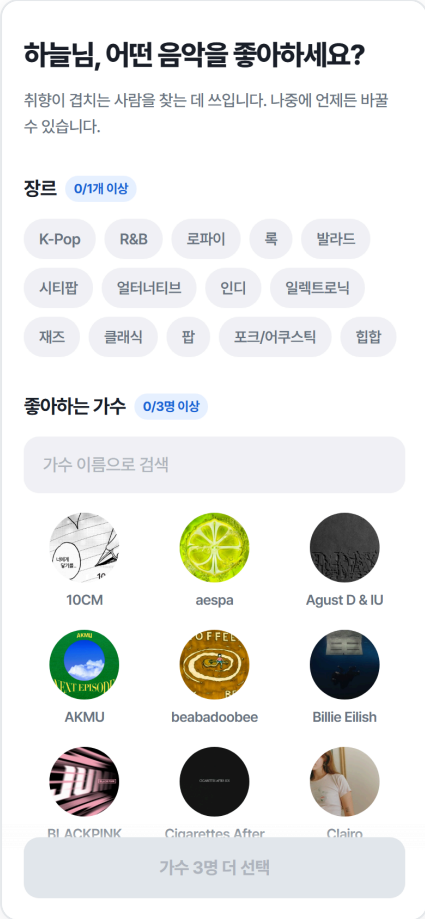
제출물

항목	내용
동작 URL	`https://soundmap.duckdns.org` — 상시 가동. 휴대폰 브라우저 권장(위치 기능은 HTTPS 필수)
소스	`잔향-소스.zip` / `github.com/HaeseokPark/Soundmap`
실행 방법	ZIP 내 `README.md` — DB 초기화 → `./gradlew bootRun` → `npm run dev`
체험 계정	로컬 실행 시 더미 40명 자동 생성 (`dummy01@dummy.soundmap.dev` ~ `dummy40@` , 비밀번호 `dummy_password`)
시연 영상	별도 제출 (720p 이상 · 5분 이내 · 설명 자막 포함)

첫 진입 화면



로그인
"같은 음악을 듣는 사람이 근처에 있습니다"



취향 온보딩
장르 1개 이상 · 가수 3명 이상

가수를 3명 이상 받는 이유

한두 명만 고르면 취향 일치도의 교집합이 0 아니면 100 으로 튀어서 숫자가 의미를 잃는다. 최소 3명을 받아야 비율이 분포를 갖는다.

취향을 등록하지 않은 사용자는 라우터가 온보딩으로 돌려보낸다. 온보딩 화면 자체는 그 검사에서 빼야 무한 리다이렉트가 나지 않는다.

저장소 구조

경로	내용
`backend/`	Spring Boot. 도메인별 패키지(auth · music · location · taste · chat · safety · settings · map · notification)
`frontend/`	React + Vite. `src/map` · `src/chat` · `src/music` · `src/pages`
`deploy/`	nginx 설정 · systemd 유닛 · 환경변수 템플릿 · 푸시 키 생성 스크립트
`README.md`	전체 API 표 · 설계 근거 · 운영 중 알게 된 함정 모음